

Лабораторна робота № 11

СТРУКТУРИ.

Мета роботи:

Вивчити поняття і застосування структур.

Обладнання та програмне забезпечення:

- IBM сумісна персональна обчислювальна машина;
- онлайн компілятор мови програмування Cі, доступний за посиланням <https://repl.it/languages/c>

Завдання до роботи:

Написати програми на мові програмування Cі, котрі використовують структури:

- для вводу інформації;
- для виводу таблиць.

Теоретичні відомості

Структура

Структура - це сукупність змінних, об'єднаних одним ім'ям, що надає загальноприйнятий спосіб спільного зберігання інформації.

Оголошення структури призводить до утворення шаблону, використовуваного для створення об'єктів структури.

Змінні, що утворюють структуру, називаються членами структури. (Члени структури також часто називаються елементами або полями.)

Зазвичай всі члени структури пов'язані один з одним. Наприклад, інформація про ім'я та адресу, що знаходиться в списку розсилки, зазвичай представляється у вигляді структури. У цьому прикладі при оголошенні структури одночасно оголошується декілька змінних.

```
struct addr {  
    char name[30];  
    char street [40];  
    char city[20];  
    char state[3];  
    unsigned long int zip;  
} addr_info, binfo, cinfo;
```

Доступ до окремих полів структури здійснюється за допомогою оператора `.` (зазвичай називається «крапка»).

Прототип:

Ім'я_структури.ім'я_поля

Наприклад, наступний фрагмент коду присвоює полю zip структурної змінної addr_info значення 79011:

```
addr_info.zip = 79011;
```

Для виводу поля zip на екран треба написати:

```
printf("%ld", addr_info.zip);
```

Присвоєння структур

Інформація, яка є в одній структурі, може бути присвоєна іншій структурі того ж типу з допомогою одиничного оператора присвоєння, тобто не потрібно присвоювати значення кожного члена (поля) окремо. Наступна програма демонструє присвоєння структур:

```
#include <stdio.h>
int main(void) {
    struct {
        int a;
        int b;
    } x, y;
    x.a = 10;
    x.b = 20;
    y = x; // присвоєння однієї структури іншій
    printf ("Contents of y: %d %d.", y.a, y.b);
    return 0;
}
```

Після присвоєння y.a і y.b будуть мати значення 10 і 20 відповідно.

Масиви структур

Для оголошення масиву структур слід спочатку визначити структуру, а потім оголосити масив змінних даного типу. Наприклад, для оголошення 100-елементного масиву структур типу addr потрібно написати:

```
struct addr addr_info[100];
```

Для доступу до окремих структур масиву addr_info потрібно проіндексувати ім'я масиву. Наприклад, для виводу змісту поля zip третьої структури, потрібно написати:

```
printf("%ld", addr_info[2].zip);
```

Слід пам'ятати, що масиви структур, як і масиви змінних, індексуються починаючи з нуля.

Вказівники на структури.

Вказівники на структуру оголошуються шляхом зазначення * перед іменем структурної змінної. Наприклад, передбачаючи, що раніше була визначена структура addr, наступна стрічка оголошує addr_pointer як вказівник на дані цього типу:

```
struct addr *addr_pointer;
```

Для отримання адреси структурної змінної потрібно розташувати оператор & перед іменем структури. Наприклад, маємо наступний фрагмент

```
struct bal {  
    float balance;  
    char name[80];  
} person;  
/* оголошення вказівника на структуру */  
struct bal *p;
```

тоді вираз

```
p = &person;
```

розташовує адресу структури person у вказівник p.

Для доступу до членів структури з допомогою вказівника на структуру потрібно використати оператор "стрілка". Оператор "стрілка", →, створений із знаку "мінус" і символа "більше". Наприклад, для доступу до поля balance з допомогою p потрібно написати:

```
p->balance;
```

Приклад використання вказівників на структури наведено в наступній простій програмі, яка виводить години, хвилини, і секунди, використовуючи програмний таймер. (Швидкість роботи таймера можна регулювати, змінюючи глобальну константу DELAY.)

```
#include <stdio.h>  
#include <conio.h>  
  
#define DELAY 128000  
  
struct my_time {  
    int hours;  
    int minutes;  
    int seconds;  
};  
void mydelay(void) {  
    for (long t = 1; t < DELAY; t++);  
}  
void update(struct my_time *t) {  
    t->seconds++;  
    if (t->seconds == 60) {  
        t->seconds = 0;  
        t->minutes++;  
    }  
    if (t->minutes == 60) {  
        t->minutes = 0;  
        t->hours++;  
    }  
    if (t->hours==24)  
        t->hours = 0;  
    mydelay();  
}
```

```

void display(struct my_time *t) {
    printf("%02d:", t->hours);
    printf("%02d:", t->minutes);
    printf("%02d\n", t->seconds);
}
int main(void) {
    struct my_time systime;
    systime.hours = 0;
    systime.minutes = 0;
    systime.seconds = 0;
    while (1) {
        update(&systime);
        display(&systime);
        if (kbhit()) return 0;
    }
}

```

Ще одним прикладом використання вказівників на структури є програма, яка ілюструє використання функції цілочисельного вводу. Функція `input_xy()` визначає координати `x` і `y`, в яких буде виводитися підказка, а потім вводить цілочисельне значення. З цією метою вона використовує структуру `xyinput`:

```

#include <stdio.h>
#include <conio.h>
#include <string.h>

struct xyinput {
    int x, y; /* положення підказки на екрані */
    char message[80]; /* підказка */
    int i; /* введене значення */
};

void input_xy(struct xyinput *info) {
    gotoxy(info->x, info->y);
    printf(info->message);
    scanf("%d", &info->i);
}

int main(void) {
    struct xyinput mess;
    mess.x = 10;
    mess.y = 10;
    strcpy(mess.message, "Enter an integer: ");
    clrscr();
    input_xy(&mess);
    printf("Your number squared is: %d.", mess.i*mess.i);
    return 0;
}

```

ПОРЯДОК ВИКОНАННЯ РОБОТИ:

1. Опрацювати і засвоїти матеріал наведений в теоретичних відомостях.
2. Написати програму з використанням структур, яка дозволяє ввести з консолі дані про 7+ гравців Вашої улюбленої спортивної команди (Прізвище | Ім'я | Зріст | Маса | Вік | Колір волосся | тощо) та вивести список команди в порядку зменшення віку спортсменів.
3. Написати програму, котра на основі структур формує і виводить таблицю з характеристиками 7+ ноутбуків (Виробник | Розмір дисплея | Роздільна здатність дисплея | Тип процесора | Розмір оперативної пам'яті | Тип накопичувача | Обсяг диска | Тип відеокарти | Ємність акумулятора | Ціна) та дозволяє користувачу обрати потрібний комп'ютер і вивести його параметри на екран.
4. В звіті навести копії екранів та написаний код.
5. Зробити висновки.